

Week 4,5 -labprogram

1. Animal Class and Objects

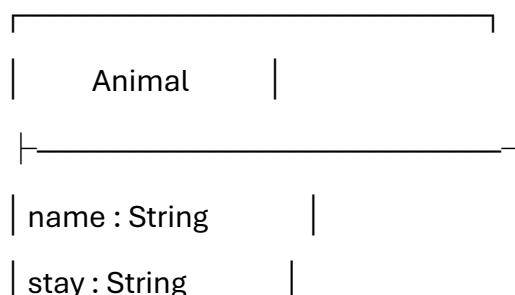
Steps

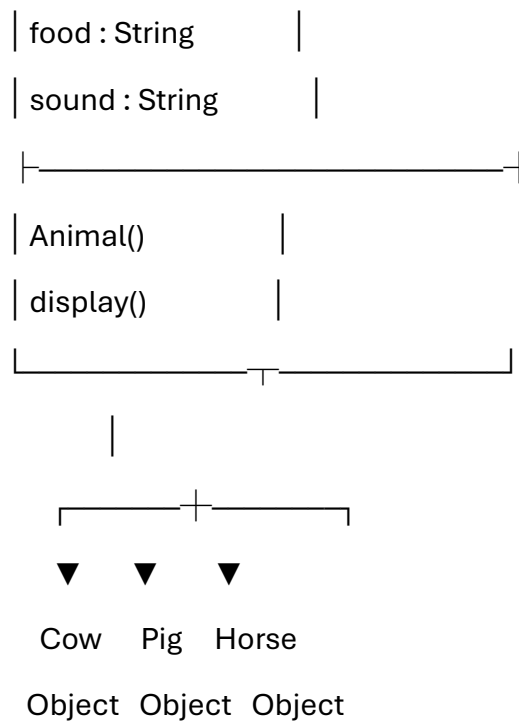
1. Create a class named Animal.
2. Declare the properties name, stay, food, and sound.
3. Create a constructor to initialize the animal details.
4. Create a display() method to display the animal details.
5. Create objects for Cow, Pig, and Horse.
6. Pass the required details through the constructor.
7. Call the display() method for each object.
8. Display the details of all animals.

Algorithm

1. Start.
2. Create the Animal class.
3. Declare animal properties.
4. Initialize the properties using a constructor.
5. Create objects cow, pig, and horse.
6. Assign values to each object.
7. Call display() for each object.
8. Display the animal name, stay, food, and sound.
9. Stop.

UML Diagram





Code

```

class Animal {
    String name;
    String stay;
    String food;
    String sound;

    Animal(String name, String stay, String food, String sound) {
        this.name = name;
        this.stay = stay;
        this.food = food;
        this.sound = sound;
    }

    void display() {
        System.out.println("Animal : " + name);
        System.out.println("Stays : " + stay);
    }
}

```

```

        System.out.println("Eats : " + food);

        System.out.println("Sound : " + sound);

        System.out.println();
    }
}

public class ClassObjLabPro {
    public static void main(String[] args) {

        Animal cow = new Animal("Cow", "Cowshed", "Grass", "Moo");
        Animal pig = new Animal("Pig", "Pigsty", "Vegetables", "Oink");
        Animal horse = new Animal("Horse", "Stable", "Grass", "Neigh");

        cow.display();
        pig.display();
        horse.display();
    }
}

```

2. Constructor Overloading

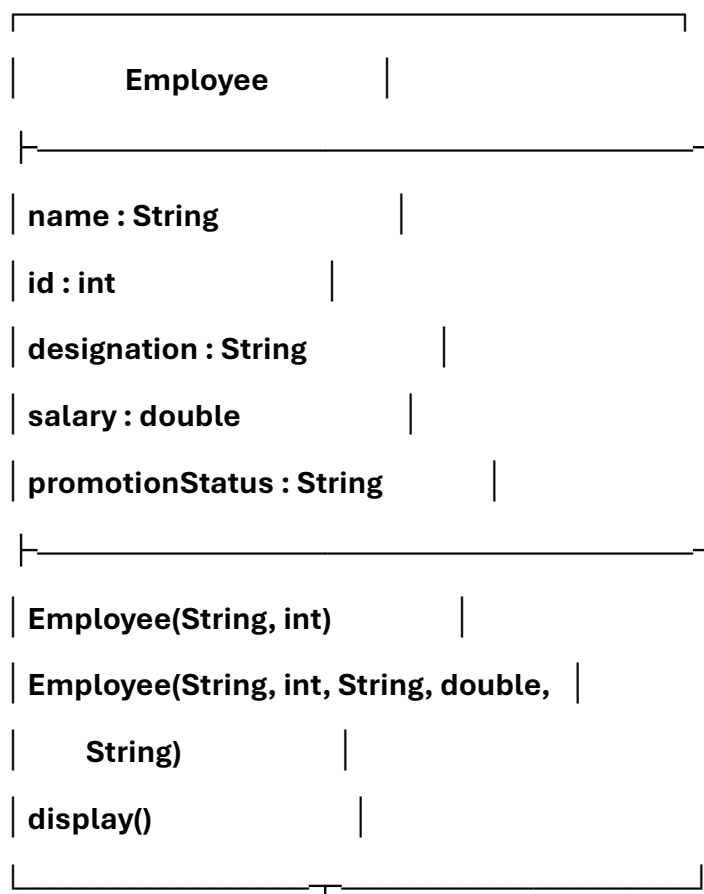
Steps

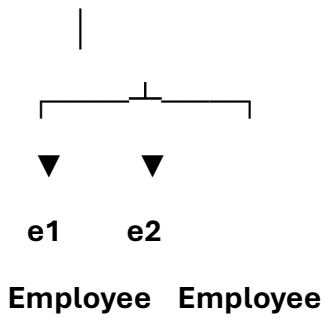
1. Create a class named Employee.
2. Declare name, id, designation, salary, and promotionStatus.
3. Create a constructor with name and id.
4. Create another constructor with all five parameters.
5. Create a display() method to display employee details.
6. Create two employee objects using different constructors.
7. Display the details of both employees.

Algorithm

1. Start.
2. Create the Employee class.
3. Declare employee details.
4. Create the first constructor with name and id.
5. Create the second constructor with name, id, designation, salary, and promotionStatus.
6. Create objects e1 and e2.
7. Initialize e1 using the first constructor.
8. Initialize e2 using the second constructor.
9. Call the display() method for both objects.
10. Display the employee details.
11. Stop.

UML Diagram





Code

```
public class Employee {  
  
    String name;  
  
    int id;  
  
    String designation;  
  
    double salary;  
  
    String promotionStatus;  
  
    Employee(String name, int id) {  
        this.name = name;  
        this.id = id;  
    }  
  
    Employee(String name, int id, String designation, double salary, String  
promotionStatus) {  
        this.name = name;  
        this.id = id;  
        this.designation = designation;  
        this.salary = salary;  
        this.promotionStatus = promotionStatus;  
    }  
  
    void display() {
```

```
        System.out.println(name);
        System.out.println(id);
        System.out.println(designation);
        System.out.println(salary);
        System.out.println(promotionStatus);
    }

    public static void main(String[] args) {
        Employee e1 = new Employee("Naveen", 101);
        Employee e2 = new Employee("Rahul", 102, "Manager", 50000, "Yes");

        e1.display();
        e2.display();
    }
}
```

```
PS C:\Users\navee\OneDrive\Desktop\javaprograms\labprograms\Week 4,5> java ClassObjLabPro
Animal : Cow
Stays  : Cowshed
Eats   : Grass
Sound  : Moo

Animal : Pig
Stays  : Pigsty
Eats   : Vegetables
Sound  : Oink

Animal : Horse
Stays  : Stable
Eats   : Grass
Sound  : Neigh
```

Naveen

101

null

0.0

null

Rahul

102

Manager

50000.0

Yes